

forum.canton.network/t/9114

****How do you prove your app didn't do something?****

Archived 2026-09-13T23:49:00Z from `https://forum.canton.network/t/9114.json`, the public Discourse endpoint. The untouched response is beside this file as `forum-9114.json`; this is a rendering of it.

4 posts, 57 views.

1. orumedominic · 2026-09-08 09:07:59 UTC

Not what it did.

Logs cover that. What it tried and was stopped from doing.

I hit this building a spend-limited agent wallet. The refusals turned out to matter more than the payments, and I could not find a good way to hand someone proof that a refusal actually happened.

What I ended up with: every attempt, allowed or refused, becomes one hash-chained receipt saying what was tried and which rule decided it.

The limits are assertMsg fences in the Daml, so the refusal is the ledger's, not my app saying so about itself. You can check a chain here with nothing installed: Check a payment record | KYA Rails

But I might be solving something nobody else has.

Has anyone ever asked you to prove your app didn't do something?

What did you show them?

2. Federico_Rodriguez · 2026-09-11 12:40:05 UTC

Hey @orumedominic , Is my understanding correct that the hash-chained receipts are produced by the application based on the response it gets from the participant/validator?

If so, one limitation is that you are still trusting the application to create the receipts and include every attempt. The hash chain makes existing receipts tamper-evident, but it doesn't prove that a receipt wasn't omitted or that its contents accurately represent the participant's response.

An alternative worth considering would be to make the refusal itself a valid Canton transaction. For example, wrap the operation in a choice that checks the policy and either performs the action or creates a RejectedAttempt audit record. The transaction commits in both cases, but the underlying action only happens when allowed.

That would make the refusal part of the Canton ledger rather than evidence produced by the application. The trade-off is that rejection becomes explicitly modeled in Daml rather than using a failed transaction.

3. orumedominic . 2026-09-13 12:04:11 UTC

(post deleted by author)

4. orumedominic . 2026-09-13 12:06:42 UTC

Yes, that is right. And the omission gap was the real problem.

I built what you described.

A choice called TryCharge checks the policy. If it passes, it pays. If it fails, it creates a ChargeRefused contract and pays nothing. Both paths commit. That is your RejectedAttempt under another name.

github.com/dominicrume/cantor8_kya_rails

step-1-mandate/daml/KyaMandate.daml

7b9eeb2d5

```
-- KyaMandate: extends the Cantor8 hackathon starter Mandate.daml (credit:
Cantor8 toolkit)
-- Additions for KYA Rails: counterparty allow-list, charge memo for the audit t
rail.
-- The cap is enforced HERE. A cap checked in Python is a suggestion.
module KyaMandate where

import DA.Time (RelTime, addRelTime)

-- One contract per accepted charge, so the audit trail lives on the ledger and
-- not only in our receipt chain. Signed by the same two parties as the mandate;
-- the payee observes, so they can read what was paid to them without being able
-- to authorise anything.
template ChargeRecord
  with
    owner      : Party
    spender    : Party
    payee      : Party
    amount     : Decimal
    memo       : Text
    chargedAt  : Time
  where
```

This file has been truncated. show original

You called the trade-off right too. Rejection is modelled in Daml now. That cost a second copy of the rules. They drifted once. I changed one comparison. Every test stayed green. The two paths then disagreed about a charge landing exactly on the cap. There is a test for that now. Your caveat still stands. An attempt nobody submits still leaves nothing. This only fixes refusals that reached the ledger. That was better review than most code gets. Thank you.
